

PERTEMUAN 5 - STACK (LIFO - LEFT IN FIRST OUT)

- Nama: Nadine Valia Azzahra
- NIM: 2C2230007
- PRODI: Sains Data

STUDI KASUS PRAKTIKUM

```
# STUDI KASUS 1: Tumpukan Buku Perpustakaan

class TumpukanBuku:
    def __init__(self):
        self.tumpukan = []

    def tambah_buku(self, judul):
        self.tumpukan.append(judul)
        print(f"Buku '{judul}' telah ditambahkan ke tumpukan teratas.")

    def ambil_buku(self):
        if self.is_empty():
            print("Tumpukan buku kosong! Tidak ada buku yang bisa diambil.")
            return None

        buku_keluar = self.tumpukan.pop()
        print(f"Buku '{buku_keluar}' telah diambil dari tumpukan teratas.")
        return buku_keluar

    def lihat_teratas(self):
        if self.is_empty():
            print("Tumpukan buku kosong.")
            return None

        print(f"[PEEK] Buku pada tumpukan teratas saat ini: '{self.tumpukan[-1]}'")
        return self.tumpukan[-1]

    def is_empty(self):
        return len(self.tumpukan) == 0

    def tampilkan_tumpukan(self):
        if self.is_empty():
            print("Keadaan tumpukan buku kosong.")
        else:
            print("\n===== TUMPUKAN BUKU SAAT INI (Top -> Bottom) =====")
            for buku in reversed(self.tumpukan):
                print(f"| [ {buku} ] |")

perpustakaan = TumpukanBuku()

perpustakaan.tambah_buku("Algoritma")
perpustakaan.tambah_buku("Struktur Data")
perpustakaan.tambah_buku("Basis Data")

perpustakaan.tampilkan_tumpukan()
perpustakaan.lihat_teratas()

perpustakaan.ambil_buku()

perpustakaan.tampilkan_tumpukan()
```

Buku 'Algoritma' telah ditambahkan ke tumpukan teratas.
 Buku 'Struktur Data' telah ditambahkan ke tumpukan teratas.
 Buku 'Basis Data' telah ditambahkan ke tumpukan teratas.

```
===== TUMPUKAN BUKU SAAT INI (Top -> Bottom) =====
| [ Basis Data ] |
| [ Struktur Data ] |
| [ Algoritma ] |
[PEEK] Buku pada tumpukan teratas saat ini: 'Basis Data'
```

Buku 'Basis Data' telah diambil dari tumpukan teratas.

```
===== TUMPUKAN BUKU SAAT INI (Top -> Bottom) =====
| [ Struktur Data ] |
| [ Algoritma ] |
```

```
# STUDI KASUS 2: Browser History (Sistem Back)

class BrowserHistory:
    def __init__(self):
        self.history = []
        self.current_page = None

    def kunjungi(self, url):
        if self.current_page is not None:
            self.history.append(self.current_page)
        self.current_page = url
        print(f"Mengunjungi halaman: {url}")

    def back(self):
        if not self.history:
            print("[BACK] Tidak ada halaman sebelumnya (riwayat kosong)!")
            return None

        page_sebelumnya = self.history.pop()
        print(f"[BACK] Kembali dari '{self.current_page}' ke '{page_sebelumnya}'")
        self.current_page = page_sebelumnya
        return self.current_page

    def tampilkan_status(self):
        print(f"\n[STATUS] Halaman saat ini : {self.current_page}")
        print(f"[RIWAYAT] Halaman di Stack : {self.history}\n")

browser = BrowserHistory()

browser.kunjungi("google.com")
browser.kunjungi("youtube.com")
browser.kunjungi("github.com")

browser.tampilkan_status()

browser.back()

browser.tampilkan_status()
```

```
Mengunjungi halaman: google.com
Mengunjungi halaman: youtube.com
Mengunjungi halaman: github.com

[STATUS] Halaman saat ini : github.com
[RIWAYAT] Halaman di Stack : ['google.com', 'youtube.com']

[BACK] Kembali dari 'github.com' ke 'youtube.com'

[STATUS] Halaman saat ini : youtube.com
[RIWAYAT] Halaman di Stack : ['google.com']
```

STUDI KASUS 3: Undo Editor Teks

```
class UndoEditorTeks:
    def __init__(self):
        self.teks = ""
        self.history_stack = []

    def ketik(self, teks_baru):
        self.history_stack.append(self.teks)
        self.teks += teks_baru
        print(f"Menambahkan kata. Teks saat ini: '{self.teks}'")

    def undo(self):
        if not self.history_stack:
            print("[UNDO] Tidak ada perubahan untuk dibatalkan (Stack Kosong)!")
            return
```

```

        self.teks = self.history_stack.pop()
        print(f"[UNDO] Berhasil dibatalkan! Teks saat ini: '{self.teks}'")

    def tampilkan_teks(self):
        print(f'\n Isi Dokumen: "{self.teks}"\n')

editor = UndoEditorTeks()

editor.ketik("Saya ")
editor.ketik("belajar ")
editor.ketik("Struktur Data")

editor.tampilkan_teks()

editor.undo()

editor.tampilkan_teks()

```

```

Menambahkan kata. Teks saat ini: 'Saya '
Menambahkan kata. Teks saat ini: 'Saya belajar '
Menambahkan kata. Teks saat ini: 'Saya belajar Struktur Data'

Isi Dokumen: "Saya belajar Struktur Data"

[UNDO] Berhasil dibatalkan! Teks saat ini: 'Saya belajar '

Isi Dokumen: "Saya belajar "

```

STUDI KASUS 4: Validasi Tanda Kurung

```

def validasi_tanda_kurung(ekspresi):
    stack = []
    pasangan_kurung = {
        ')': '(',
        '}': '{',
        ']': '['
    }

    for karakter in ekspresi:
        if karakter in "({[":
            stack.append(karakter)

        elif karakter in ")}]":
            if not stack or stack[-1] != pasangan_kurung[karakter]:
                return False

            stack.pop()

    return len(stack) == 0

ekspresi = "{[()]}"

if validasi_tanda_kurung(ekspresi):
    print("Ekspresi VALID")
else:
    print("Ekspresi TIDAK VALID")

```

Ekspresi VALID

STUDI KASUS 5: Riwayat Transaksi ATM

```

class RiwayatTransaksiATM:
    def __init__(self, limit=5):
        self.stack_transaksi = []
        self.limit = limit

    def catat_transaksi(self, jenis, nominal):
        deskripsi = f"{jenis} sebesar Rp {nominal:,.}"

        if len(self.stack_transaksi) >= self.limit:
            self.stack_transaksi.pop(0)

        self.stack_transaksi.append(deskripsi)
        print(f"Transaksi tercatat: {deskripsi}")

```

```

def tampilkan_transaksi_terakhir(self):
    if not self.stack_transaksi:
        print("Belum ada riwayat transaksi.")
        return

    print(f"\n===== {len(self.stack_transaksi)} TRANSAKSI TERAKHIR (Terbaru -> Terlama) =====")

    for i, trx in enumerate(reversed(self.stack_transaksi), 1):
        print(f"{i}. {trx}")

atm = RiwayatTransaksiATM()

atm.catat_transaksi("Setor Tunai", 500000)
atm.catat_transaksi("Tarik Tunai", 200000)
atm.catat_transaksi("Transfer", 100000)
atm.catat_transaksi("Pembayaran", 150000)

atm.tampilkan_transaksi_terakhir()

```

```

Transaksi tercatat: Setor Tunai sebesar Rp 500,000
Transaksi tercatat: Tarik Tunai sebesar Rp 200,000
Transaksi tercatat: Transfer sebesar Rp 100,000
Transaksi tercatat: Pembayaran sebesar Rp 150,000

===== 4 TRANSAKSI TERAKHIR (Terbaru -> Terlama) =====
1. Pembayaran sebesar Rp 150,000
2. Transfer sebesar Rp 100,000
3. Tarik Tunai sebesar Rp 200,000
4. Setor Tunai sebesar Rp 500,000

```